

C++

Programming:

An Object-Oriented Approach

Chapter 12

Polymorphism and Other Issues

C++

Programming:

An Object-Oriented
Approach



BEHROUZ A. FOROUZAN
RICHARD F. GILBERG

BEHROUZ A. FOROUZAN
RICHARD F. GILBERG

Objectives Part 1

To define polymorphism and describe three necessary conditions to achieve it.

To define virtual tables and how they help the system to decide which virtual function to use during runtime.

To discuss virtual destructors and why they are needed for the proper use of polymorphism.

To distinguish between static and dynamic binding in inheritance.

To discuss Run Time Type Information (RTTI) and how it is used.

Objectives Part 2

To introduce abstract classes and their use.

To introduce interface classes and their use.

To discuss multiple inheritance and its associated problems.

To show how the use of virtual base inheritance and mixin classes can eliminate the problem of duplicated data members in multiple inheritance.

POLYMORPHISM PART 1

One of the main pillars of object-oriented programming is polymorphism.

Polymorphism gives us the ability to write several versions of a function, each in a separate class.

Then, when we call the function, the version appropriate for the object being referenced is executed.

POLYMORPHISM PART 2

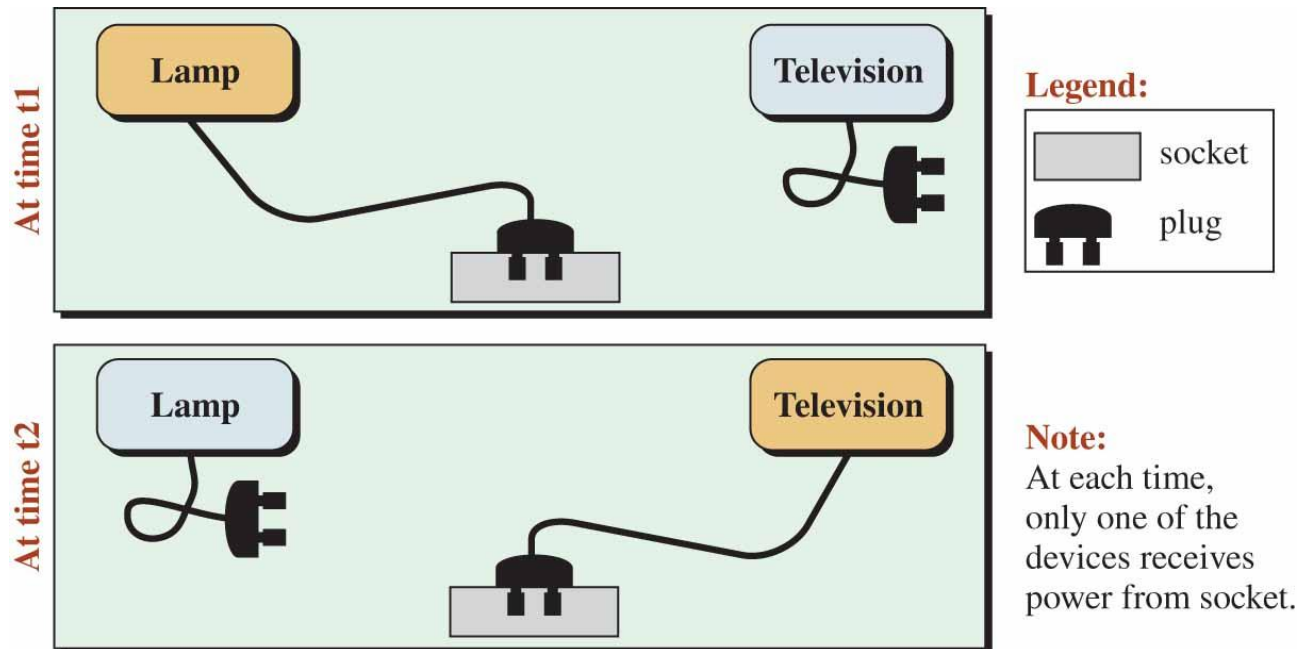
We use one verb (function) to mean different things.

For example, we say "open" meaning to open a door, a jar, or a book; which one is determined by the context. Similarly, in C++, we can call a function named *printArea* to print the area of a triangle or the area of a rectangle.

POLYMORPHISM PART 3

To understand the concept of polymorphism, we need to first understand the concept of plug-compatible objects.

Figure 12.1 *A socket and plug-compatible devices*



[Access the text alternative for slide images.](#)

POLYMORPHISM PART 4

Assume we have two electrical devices (such as a table lamp and a television set).

We have only one socket that can supply power to one of these devices at a time.

Each device has a plug that can be inserted in the only socket.

At time t_1 , we plug the lamp into the socket.

At time t_2 , we unplug the lamp and plug the TV set in the socket.

POLYMORPHISM PART 5

We can do so because the two devices are plug-compatible; their plugs follow the same standard.

The interesting point about plug-compatible devices is that all get the same thing (electrical power) from the socket, but each does a different tasks.

Condition for Polymorphism₁

We define the conditions for polymorphism in inheritance by comparing it with the plug-compatible analogy.

Pointer or References.

To follow the analogy, we need to have something that plays the role of a socket that once created can accept plug-compatible objects.

In C++, a pointer or a reference can play this role. We can define a pointer (or a reference) that can point to the base class; we can then let the pointer point to any object in the hierarchy.

For this reason, the pointer and reference variables are sometimes referred to as polymorphic variables.

Condition for Polymorphism₂

Exchangeable Objects.

To continue with the analogy, we need to have plug-compatible objects that play the role of the devices.

An object in an inheritance hierarchy plays this role.

Condition for Polymorphism₃

Virtual Functions.

We need something to play the role of power given to different devices that perform different tasks.

This is done in C++ using virtual functions, that are modified by the keyword virtual.

For example, we can have a print function in all classes, all named the same but each prints differently.

For polymorphism, we need pointers (or references), we need exchangeable objects, and we need virtual functions.

Condition for Polymorphism₄

Example 12.1

Program 12-2 shows an incomplete polymorphic program using only the first two conditions.

We define two classes. We then create a pointer (simulating a socket) that can accept an object of each class at different times (plug-compatible object).

For simplicity, we define only one public member function for each class and let the system add default constructors.

Incomplete Polymorphic Program Part 1

Program 12.1 *An incomplete polymorphic program*

```
1  /*****
2  * A simple program to show the first two conditions for      *
3  * polymorphism                                              *
4  *****/
5  #include <iostream>
6  #include <string>
7  using namespace std;
8
9  // Definition of Base class and in-line print function
10 class Base
11 {
12     public:
13         void print () const {cout << "In the Base" << endl;}
14 };
15 // Definition of Derived class and in-line print function
16 class Derived : public Base
17 {
18     public:
19         void print () const {cout << "In the Derive" << endl;}
20 };
```

Incomplete Polymorphic Program Part 2

Program 12.1 An incomplete polymorphic program

```
21
22 int main ( )
23 {
24     // Creation of a pointer to the Base class (simulating socket)
25     Base* ptr;
26     // Let ptr points to an object of the Base class
27     ptr = new Base ( );
28     ptr -> print();
29     delete ptr;
30     // Let ptr points to an object of the Derived class
31     ptr = new Derived();
32     ptr -> print();
33     delete ptr;
34     return 0;
35 }
```

Run:

In the Base
In the Base

Incomplete Polymorphic Program Part 3

At line 27 *ptr* is pointing to an object of the Base class and at line 28 we call the function defined in the *Base* class.

At line 31, we make the same pointer point to an object of the *Derived* class and at line 32 we tried to call the function defined in the *Derived* class, but the result shows the function defined for the *Base* class is called.

The reason is that while the first two conditions of polymorphism are accomplished in the program, the third condition (virtual functions) is not fulfilled.

The print function is not a virtual function.

Incomplete Polymorphic Program Part 4

The result should be expected because the variable *ptr* is defined as pointer to *Base*.

It can accept being pointed to an object of the *Derived* type because a derived object is a *Base* (inheritance defines an *is-a* relationship).

However, when it wants to call the print function, it is still a pointer to *Base*, so it calls the print function defined in the *Base* class.

We have not changed the type of the pointer, we have just forced it to point to the *Derived* class.

Polymorphic Program

Example 12.2

We repeat the previous example, but we make the print function virtual.

The result is that the correct function is activated in each call as shown in Program 12-2.

Polymorphic Program Using All Three Conditions 1

Program 12.2 A valid polymorphic program using all three conditions

```
1  /*****
2  * A simple program to show that if all three necessary      *
3  * conditions are fulfilled, we have polymorphism            *
4  *****/
5  #include <iostream>
6  #include <string>
7  using namespace std;
8
9  // Definition of Base class and in-line definition for print function
10 class Base
11 {
12     public:
13         virtual void print () const {cout << "In the Base" << endl;}
14 };
15 // Definition of Derived class and in-line definition for print function
16 class Derived : public Base
17 {
18     public:
19         virtual void print () const {cout << "In the Derive" << endl;}
20 };
```

Polymorphic Program Using All Three Conditions 2

Program 12.2 *A valid polymorphic program using all three conditions*

```
21
22 int main ( )
23 {
24     // Creation of a pointer to the Base class (simulating socket)
25     Base* ptr;
26     // Let ptr points to an object of the Base class
27     ptr = new Base ();
28     ptr -> print();
29     delete ptr;
30     // Let ptr points to an object of the Derived class
31     ptr = new Derived();
32     ptr -> print();
33     delete ptr;
34     return 0;
35 }
```

Run:

In the Base

In the Derived

Need of Virtual Modifier

Although we have added the virtual modifier to both print functions, it is not needed.

When a function is defined as virtual, all functions in the hierarchy of classes with the same signature are automatically virtual.

Mechanism

To understand how virtual functions take part in polymorphic behavior, we need to understand virtual tables (*vtables*).

In polymorphism, the system creates a virtual table for each class in the hierarchy of classes.

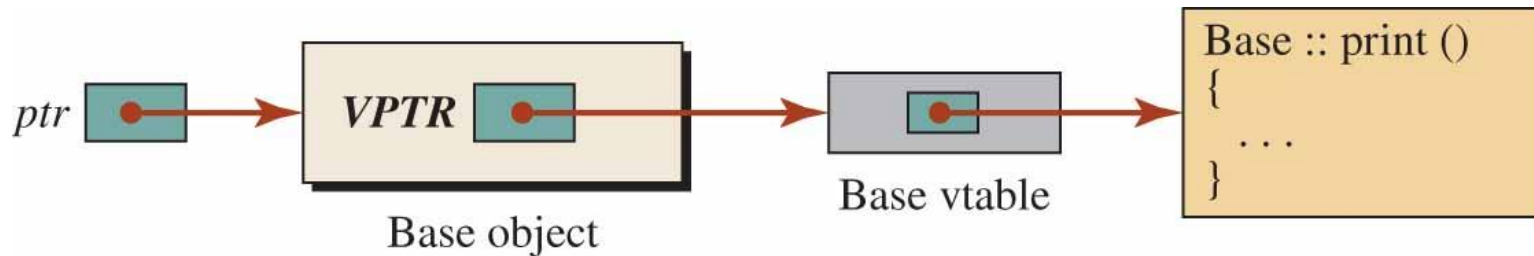
Each entry in each *vtable* has a pointer to the corresponding virtual function.

Each object created in an application will have an extra data member which is a pointer to the corresponding *vtable*.

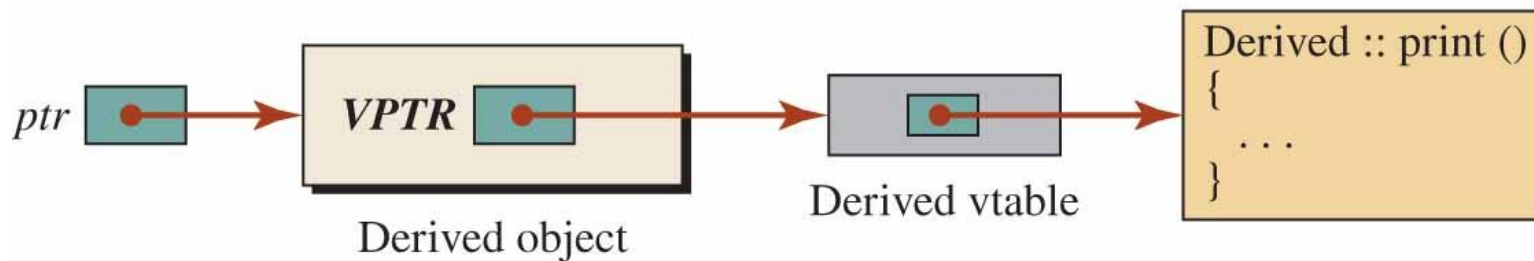
In the case of our simple program, there are two objects and two *vtables* each with one entry as shown in Figure 12.2.

Virtual Tables for Base and Derived Classes Part 1

Figure 12.2 *Virtual tables for Base and Derived classes*



When *ptr* is pointing to the Base class



When *ptr* is pointing to the Derived class

Virtual Tables for Base and Derived Classes Part 2

When *ptr* is pointing to the *Base* object, the *VPTR* pointer added to the *Base* class object is reached, which is pointing to the *vtable* of the *Base* class.

In this case, the *vtable* has only one entry which invokes the only virtual function in *Base* class.

When *ptr* is pointing to the *Derived* object, the *vtable* of the *Derived* object is reached and the print function defined in the *Derived* class is invoked.

Constructors and Destructors₁

Constructors and destructors in a class hierarchy are also member functions although special ones.

We discuss them separately.

Constructor Cannot Be Virtual.

Constructors cannot be virtual because although constructors are member function, the names of the constructors are different for the base and derived classes (different signatures).

Constructors and Destructors₂

Virtual Destructor.

Although the name of the destructors are also different in the base and derived class, the destructors are normally not called by their name.

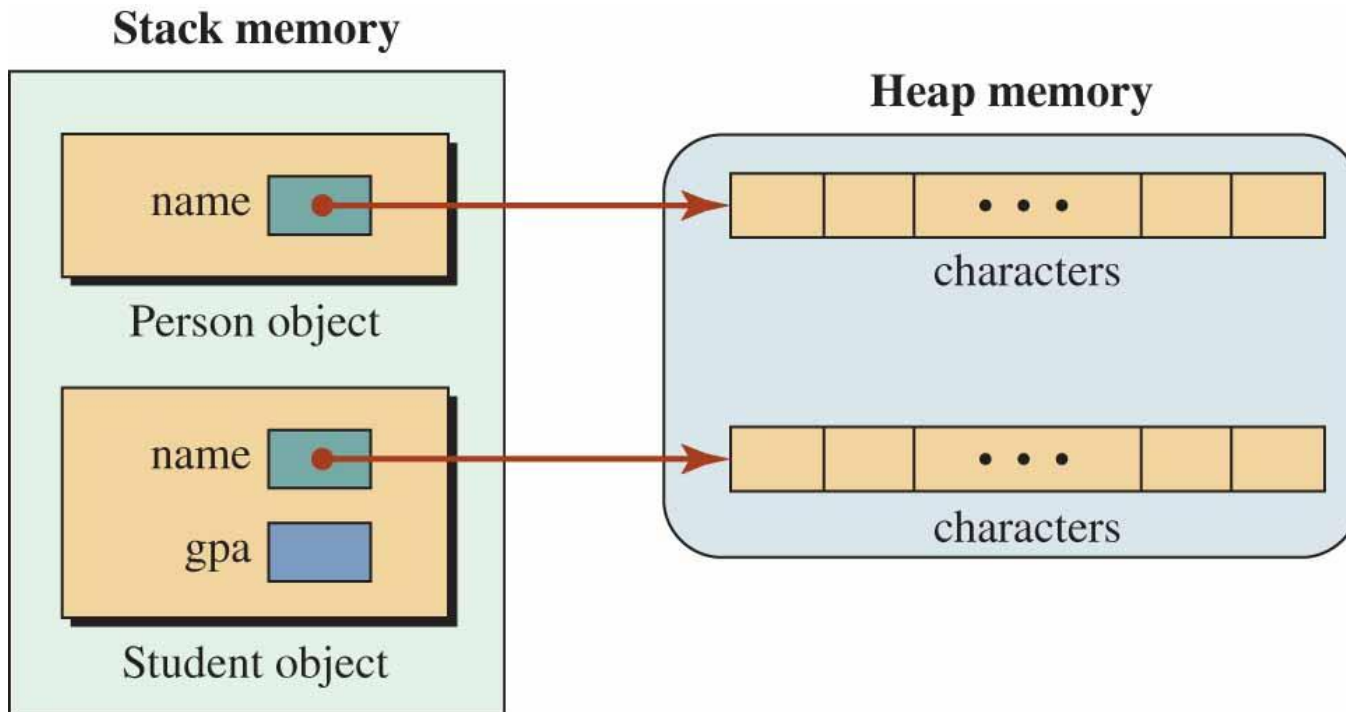
When there is a virtual member function anywhere in the design, we should make the destructors also virtual to avoid memory leaks.

Case 1: No Polymorphism₁

When we are not using polymorphism.

We create a *Person* class and a *Student* class.

Figure 12.3 *Two objects in a program not using polymorphism*



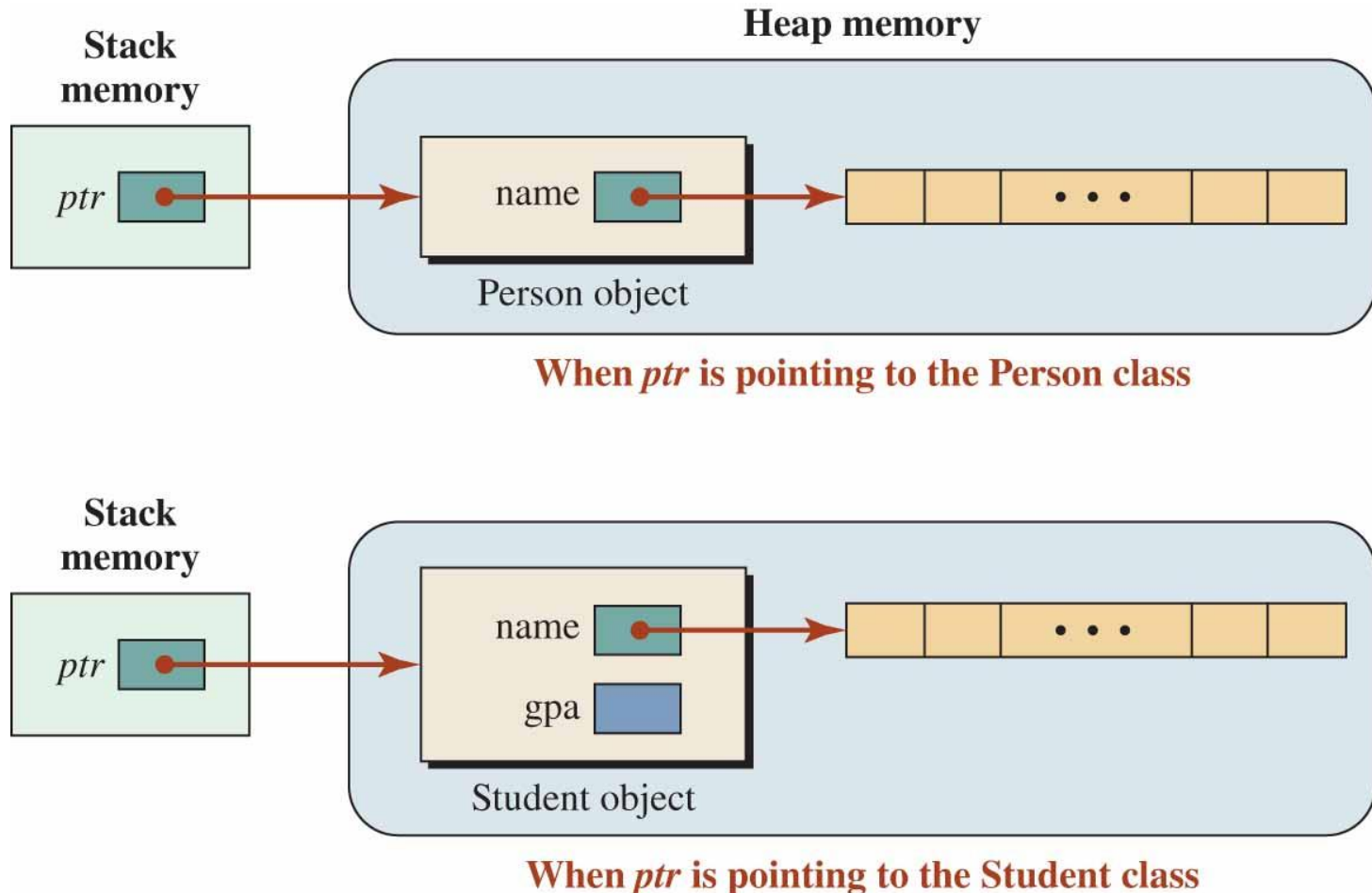
Case 1: No Polymorphism₂

We cannot have a memory leak in this situation.

When the program terminates the destructor for Person class and Student class are called, which automatically calls the destructors of the string class, which delete the allocated memory in the heap.

Case 2: Polymorphism Part 1

Figure 12.4 *Two objects in a program using polymorphism*



Case 2: Polymorphism Part 2

A problem may be created when ptr is deleted as shown below:

```
ptr = new Person(...);  
...  
delete ptr; // It deletes Person because ptr type is Person*  
  
ptr = new Student(...);  
...  
delete ptr; // It does not deletes Student because ptr type is Person*
```

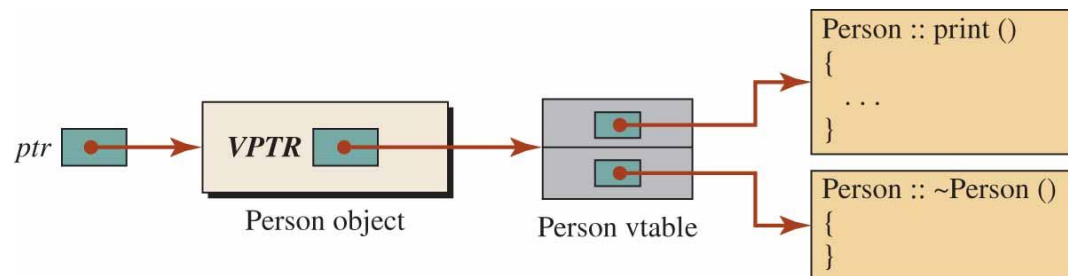
The solution is to make the destructor of the base class *virtual*, which automatically makes the destructor of the derived class *virtual*.

In this case, the system allows two different members functions with different names to be virtual and they are both added to the virtual table.

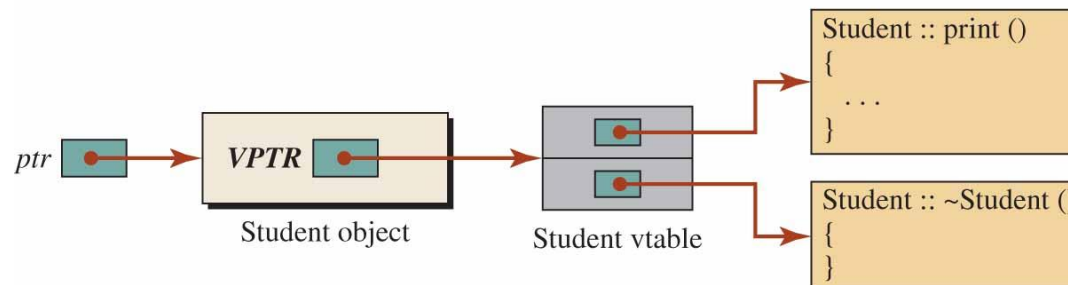
Case 2: Polymorphism Part 3

Figure 12.5 shows that we have two entries in the virtual table and when an object in the heap is deleted, the program knows which destructor should be called.

Figure 12.5 *Virtual tables when using virtual destructors*



When *ptr* is pointing to the **Person** class



When *ptr* is pointing to the **Student** class

Case 2: Polymorphism Part 4

C++ recommends that we always define an explicit destructor for the base class in polymorphism and make it virtual. Use of virtual destructors prevents possible memory leaks in polymorphism.

Example

We show how we can use polymorphism to print the information in the *Student* class through a pointer pointing to the *Person* class.

Interface File for the Person Class Part 1

Program 12.3 File *person.h*

```
1  /*****
2  * The interface file for the Person class *
3  *****/
4  #ifndef PERSON_H
5  #define PERSON_H
6  #include <iostream>
7  #include <string>
8  using namespace std;
9
10 class Person
11 {
12     private:
13         string name;
14     public:
15         Person (string name);
16         virtual ~Person ();
17         virtual void print () const;
18 };
19 #endif
```


Interface File for the Person Class Part 2

Program 12.4 File *person.cpp*

```
1  /*****
2   * The implementation file for the Person class          *
3   *****/
4  #include "Person.h"
5
6  // Definition of the Person constructor
7  Person :: Person (string nm)
8  : name (nm)
9  {
10 }
11 // Definition of the Person destructor (virtual)
12 Person :: ~Person ()
13 {
14 }
15 // Definition of the print function (virtual)
16 void Person :: print () const
17 {
18     cout << "Name: " << name << endl;
19 }
```

Interface File for the Person Class Part 3

Program 12.5 File student.h

```
1  /*****
2  * The interface file for the Student class
3  *****/
4  #ifndef STUDENT_H
5  #define STUDENT_H
6  #include "person.h"
7
8  class Student: public Person
9  {
10     private:
11         double gpa;
12     public:
13         Student (string name, double gpa);
14         virtual void print () const;
15 };
16 #endif
```

Implementation File for the Person Class₁

Program 12.6 File student.cpp

```
1  /*****
2   * The implementation file for the Student class          *
3   *****/
4  #include "Student.h"
5
6  // Definition of Constructor for Student class
7  Student :: Student (string nm, double gp)
8  : Person (nm), gpa (gp)
9  {
10 }
11 // Definition of virtual print function for Student class
12 void Student :: print () const
13 {
14     Person :: print ();
15     cout << "GPA: " << gpa << endl;
16 }
```

Application File for the Person Class Part 1

Program 12.7 File *app.cpp*

```
1  /*****
2  * The application file to test Person and Student classes      *
3  *****/
4  #include "Student.h"
5
6  int main ( )
7  {
8      // Creation of ptr as polymorphic variable
9      Person* ptr;
10     // Instantiation Person object in the heap
11     ptr = new Person ("Lucie");
12     cout << "Person Information";
13     ptr -> print();
14     cout << endl;
15     delete ptr;
16     // Instantiation Student object in the heap
17     ptr = new Student ("John", 3.9);
18     cout << "Student Information";
19     ptr -> print();
20     cout << endl;
```

Application File for the Person Class Part 2

Program 12.7 File *app.cpp*

```
21     delete ptr;  
22     return 0;  
23 }
```

Run:

```
Person Information  
Name: Lucie  
Student Information  
Name: John  
GPA: 3.9
```

Better Use of Polymorphism Part 1

We actually did not need polymorphism in previous programs.

We can achieve the same result without using a pointer to point to different objects.

We can use *person.print ()* instead of *ptr -> print()* and we can use *student.print ()* instead of *ptr -> print()*.

However, the program shows the idea of polymorphism with only one pointer that can point to different objects.

Better Use of Polymorphism Part 2

A better demonstration is when we have to use polymorphism. Assume we need to have an array of objects.

We know that all elements of an array needs to be of the same type; this means we cannot use an array of objects if the objects are of different types.

However, we can use an array of pointers, in which each pointer can point to an object of the base class (*Person* in the previous example).

In other words, we can have an array of polymorphic variables, instead of one.

Program 12-8 is the same as Program 12-7 but we have used an array of pointers.

Using an array of Pointers in Polymorphism Part 1

Program 12.8 Using an array of pointers in polymorphism

```
1  /*****
2  * Modification of application file to show the actual use of *
3  * polymorphism with an array of pointers. *
4  *****/
5  #include "student.h"
6
7  int main ( )
8  {
9      // Declaration of an array of polymorphic variables (pointers)
10     Person* ptr [4];
11     // Instantiation of four objects in the heap memory
12     ptr[0] = new Student ("Joe", 3.7);
13     ptr[1] = new Student ("John", 3.9);
14     ptr[2] = new Person ("Bruce");
15     ptr[3] = new Person ("Sue");
16     // Calling the virtual print function for each object
17     for (int i = 0; i < 4; i++)
18     {
19         ptr[i] -> print ();
20         cout << endl;
```


Using an array of Pointers in Polymorphism Part 2

Program 12.8 Using an array of pointers in polymorphism

```
21     }  
22     // Deleting the objects in the heap  
23     for (int i = 0; i < 4; i++)  
24     {  
25         delete ptr [i];  
26     }  
27     return 0;  
28 }
```

Run:

Name: Joe

GPA: 3.70

Name: John

GPA: 3.90

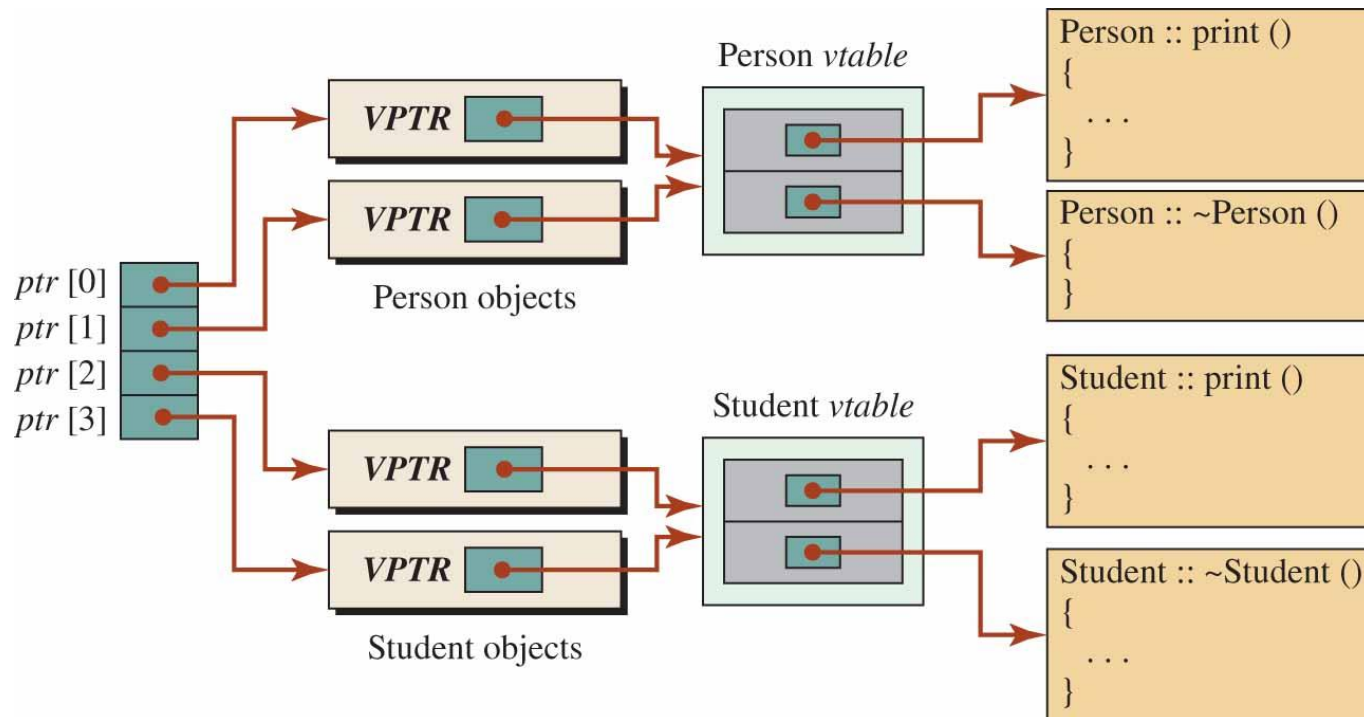
Name: Bruce

Name: Sue

Virtual Tables of Objects

Figure 12.6 shows the virtual table for the above program in which four objects, two of each type.

Figure 12.6 *Virtual tables of objects in Program 12.8*



Polymorphism In Other Languages Part 1

These three conditions we mentioned for polymorphism in C++ exist in other languages, but they may be hidden.

For example, in the Java language, which was designed after C++, we have polymorphism with all of the above three conditions.

Polymorphism In Other Languages Part 2

- a.** Java supports public inheritance (in fact, Java has only one type of inheritance, public), which means we have interchangeable objects through inheritance.
- b.** Although there are no pointers or references in Java at the user level, class objects are always created in the heap and there are variables (called reference types) that points to these objects; each class object is accessible through polymorphic variables (sockets).
- c.** Finally, all member functions in Java are virtual functions by default, which means that the third condition, existence of polymorphic-function, is also fulfilled.

Binding Part 1

An issue related to polymorphism that needs to be discussed is *binding*.

We know that a function is split into two entities: function call and function definition.

Binding here means the association between the function call for example, *print ()*, and the function body, for example, *void print {...}*.

We know that the function definition is created somewhere and the function call occurs somewhere else.

If we have only one single definition for a function, there is no confusion. Whenever we call a function the corresponding definition is executed.

Binding Part 2

We know that we may have two functions with the same signature (overriding functions). This means that the function has only one form of call, but may have more than one definition.

If a Person object calls the print function, one definition is executed; if a Student object calls the print function, a different definition is executed.

Binding here means how the program binds (associates) a function call to a function definition. There are two cases: *static binding* and *dynamic binding*.

Static Binding Part 1

The term *static binding* (sometimes called *compile-time binding* or *early binding*) occurs when we have more than one definition for a function, but the compiler exactly knows which version of the definition is to be used when the program is compiled.

This may happens, for example, when the function is called by its corresponding object as show below.

```
person.print();  
student.print();
```

Static Binding Part 2

When the compiler encounters the first call, it knows that the call should be for the definition of print function that prints the data member of the Person object.

When the compiler encounters the second call, it knows that the call should be to the definition of the print function that prints the data members of the Student object.

Dynamic Binding

There is no ambiguity and there is no need for polymorphism. Everything is defined earlier; binding is established during compilation time and the compiler establishes the association.

We use polymorphism for *dynamic binding* (also called *late binding* or *run-time binding*), which means that we need to bind a call to the corresponding definition during run time.

This is needed when the object is not known during the compilation.

For this reason, we need a virtual function to force the run-time system to create a table that shows which object needs which function, and bind the call to the appropriate function.

Polymorphism is closely tied to dynamic binding because we want to be able to execute the appropriate function definition.

Run-Time Type Information (RTTI) Part 1

When working with hierarchy of classes, sometimes we need to know the type of the object we are dealing with or sometimes we want to change the type of the object.

Using typeid Operator.

To determine the type of the object at run time, we can use the `<typeinfo>` header to access an object of the class *type_info*.

It has no constructor, destructor, or copy constructor.

We create an object of *type_info* using an overloaded operator named *typeid*.

We can create an object of *type_info* by passing an expression to the operator *typeid* that can be evaluated as a type.

Run-Time Type Information (RTTI) Part 2

For example *typeid* (5), *typeid* (object_name), *typeid* (6 + 2), and so on.

We can then use one of the four member functions or operators that are defined in the *type_info* class as shown in Table 12.1 in which *t1* and *t2* are object of *type_info* class.

Table 12.1 Operation on *type_info* objects

```
t1 == t2           // Returns true if t1 and t2 are of the same type
t1 != t2           // Returns true if t1 and t2 are of different types
t1.name()          // Returns a C-type string (name of the t1)
t1.before(t2)      // Returns true if t1 comes before t2
```

Testing typeid Operator Part 1

Program 12.9 Testing typeid operator

```
1  /*****
2  * A program to use typeid operator to find the name of classes *
3  *****/
4  #include <iostream>
5  #include <string>
6  #include <typeinfo>
7  using namespace std;
8
9  class Animal {};
10 class Horse {};
11
12 int main ( )
13 {
14     Animal a;
15     Horse h;
16     cout << typeid(a).name() << endl;
17     cout << typeid(h).name();
18     return 0;
19 }
```

Testing typeid Operator Part 2

Program 12.9 *Testing typeid operator*

Run:

6Animal

5Horse

The name of the class is preceded by the number of character in each case (6 and 5).

Using Dynamic-Cast Operator

We have seen that in a polymorphic relationship we can *upcast* a pointer, which means to make a pointer to a derived class to be assigned to a pointer to a base class as shown below:

```
Person* ptr1 = new Student
```

Here the pointer returned from the *new* operator is a pointer to a *Student* object, but we assign it to a pointer that points to a *Person* object (the pointer is upcast).

C++ also allows us to downcast a pointer to make it to point to an object in the lower order of hierarchy.

This can be done using a *dynamic_cast* operator as shown below (*ptr1* is a pointer to *Person* object).

12.1.4 Continued

```
Student* ptr2 = dynamic_cast <Student*> (ptr1) ;
```

This casting proves that the *Student* class is a class derived from the *Person* class because *ptr1* can be downcast to *ptr2*.

However it is not recommended because of the overhead involved.

OTHER ISSUES

In this section, we discuss abstract classes and multiple inheritance.

We include a project involving several levels of inherited classes.

Abstract Classes Part 1

When we design a set of classes, sometimes we find that there is a list of behaviors that are identical to all classes.

For example, assume we define two classes named *Rectangle* and *Square*.

Both of these classes have at least two common behaviors: *getArea()* and *getPerimeter()*.

How can we force the creator of these two classes (in particular when each class is created by a different entity) to provide the definition of both member functions for each class?

We know that the formula to find the area and perimeter of these geometrical shapes is different; which means that each class must have its own version of *getArea ()* and *getPerimeter ()*.

Abstract Classes Part 2

The solution in object-oriented programming is to declare an *abstract* class, that forces the creators of all derived classes to remember to add these two definitions to their classes.

A set of classes with one abstract class must have the declaration and definition for the *pure virtual functions*.

Abstract Classes Part 3

An *abstract class* is a class with at least one *pure virtual function*.

Declaration of Pure Virtual Functions.

An *abstract class* is a class with at least one *pure virtual function*.

A *pure virtual function* is a virtual function in which the declaration is set to zero and which has no definition in the *abstract class*.

The following shows two *virtual member function* for the *Shape* class:

```
virtual double getArea(0) = 0;  
virtual double getPerimeter(0) = 0;
```

Abstract Classes Part 4

Definition of Pure Virtual Functions.

The abstract class does not define its pure virtual function, but every class that inherits from the abstract class needs to provide the definition of each pure virtual function or declared it as a pure virtual to be defined in the next lower level of the hierarchy.

No Instantiation

We cannot instantiate an object from an abstract class because it does not have the definition of its pure virtual functions.

For an object to be able to instantiated from a class, the class needs to have the definition of all member function.

This means that an abstract class needs to be polymorphically inherited to define concrete classes for instantiation.

An abstract class cannot be instantiated because there is no definition for the pure virtual member functions.

Interfaces Part 1

An abstract class can have both virtual and pure virtual functions.

In some cases, however, we may need to create a blueprint for inherited classes.

We can define a class with all pure virtual functions.

This class is sometimes referred to as an interface; we cannot create any implementation file from this class, only the interface file.

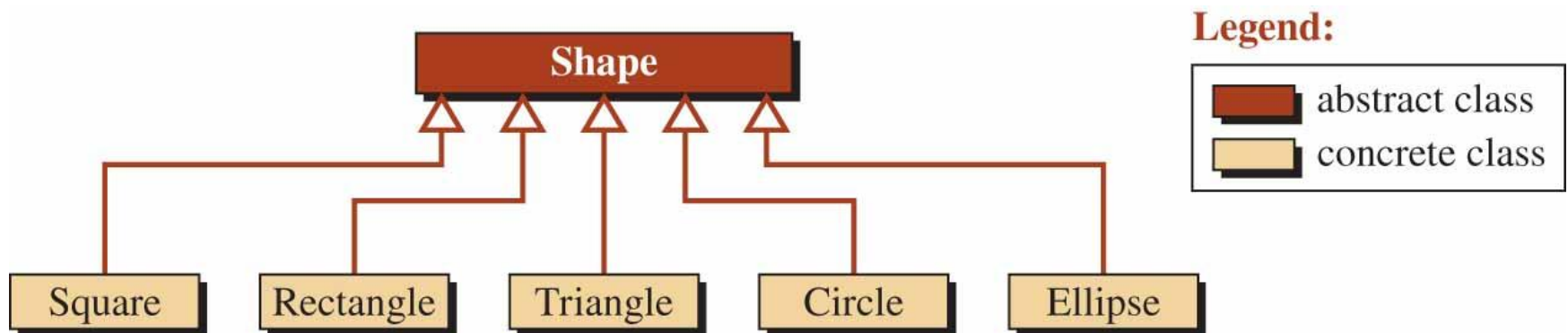
Interfaces Part 2

An interface is a special case of an abstract class in which all member functions are pure virtual.

Example

We create five concrete classes to represent shapes. All classes are inherited from an abstract class Shape.

Figure 12.7 *Adding an abstract class to a set of classes*



Interface for the Abstract Shape Class

Part 1

Program 12.10 File *shape.h*

```
1  /*****
2  * The interface for the abstract Shape class *
3  *****/
4  #ifndef SHAPE_H
5  #define SHAPE_H
6  #include <iostream>
7  #include <cassert>
8  #include <cmath>
9  using namespace std;
10
11 // Class definition
12 class Shape
13 {
14     protected:
15         virtual bool isValid () const = 0;
16     public:
17         virtual void print () const = 0 ;
18         virtual double getArea () const = 0 ;
19         virtual double getPerimeter () const = 0;
20 };
```


Interface for the Abstract Shape Class

Part 2

Program 12.10 *File shape.h*

```
21 #endif
```

Interface File of the Square Class Part 1

Program 12.11 File square.h

```
1  /*****
2  * The interface file the Square class *
3  *****/
4  #ifndef SQUARE_H
5  #define SQUARE_H
6  #include "shape.h"
7
8  // Class Definition
9  class Square : public Shape
10 {
11     private:
12         double side;
13         bool isValid() const;
14     public:
15         Square (double side);
16         ~Square ();
17         void print() const;
18         double getArea () const;
19         double getPerimeter () const;
20 };
```

Interface File of the Square Class Part 2

Program 12.11 *File square.h*

```
21 #endif
```

Implementation File of the Square Class

Part 1

Program 12.12 File *square.cpp*

```
1  /*****
2  * The implementation file the Square class *
3  *****/
4  #include "square.h"
5
6  // Constructor
7  Square :: Square (double s)
8  :side (s)
9  {
10     if (!isValid ())
11     {
12         cout << "Invalid square!";
13         assert (false);
14     }
15 }
16 // Destructor
17 Square :: ~Square ()
18 {
19 }
20 // Definition of print function
```

Implementation File of the Square Class

Part 2

Program 12.12 *File square.cpp*

```
21 void Square :: print () const
22 {
23     cout << "Square of side " << side << endl;
24 }
25 // Finding the area
26 double Square :: getArea () const
27 {
28     return (side * side);
29 }
30 // Finding the perimeter
31 double Square :: getPerimeter () const
32 {
33     return (4 * side);
34 }
35 // Private isValid function
36 bool Square :: isValid () const
37 {
38     return (side > 0.0);
39 }
```

Interface File for the Rectangle Class

Part 1

Program 12.13 File *rectangle.h*

```
1  /*****
2  * The interface file for the Rectangle class          *
3  *****/
4  #ifndef RECTANGLE_H
5  #define RECTANGLE_H
6  #include "shape.h"
7
8  // Class definition
9  class Rectangle : public Shape
10 {
11     private:
12         double length;
13         double width;
14         bool isValid() const;
15     public:
16         Rectangle (double length, double width);
17         ~Rectangle ();
18         void print () const;
19         double getArea() const;
20         double getPerimeter() const;
```

Interface File for the Rectangle Class

Part 2

Program 12.13 *File rectangle.h*

```
21 }  
22 #endif
```

Implementation File the Rectangle Class

Part 1

Program 12.14 File *rectangle.cpp*

```
1  /*****
2  * The implementation file the Rectangle class          *
3  *****/
4  #include "rectangle.h"
5
6  // Constructor
7  Rectangle :: Rectangle (double lg, double wd)
8  : length (lg), width (wd)
9  {
10     if (!isValid())
11     {
12         cout << "Invalid rectangle!";
13         assert (false);
14     }
15 }
16 // Destructor
17 Rectangle :: ~Rectangle ()
18 {
19 }
20 // Definition of print function
```


Implementation File the Rectangle Class

Part 2

Program 12.14 File *rectangle.cpp*

```
21 void Rectangle :: print () const
22 {
23     cout << "Rectangle of " << length << " X " << width << endl;
24 }
25 // Finding the area
26 double Rectangle :: getArea() const
27 {
28     return length * width;
29 }
30 // Finding the perimeter
31 double Rectangle :: getPerimeter() const
32 {
33     return 2 * (length + width);
34 }
35 // Private isValid function
36 bool Rectangle :: isValid () const
37 {
38     return (length > 0.0 && width > 0.0);
39 }
```

Interface File for the Triangle Class Part 1

Program 12.15 File triangle.h

```
1  /*****
2  * The interface file for the Triangle class *
3  *****/
4  #ifndef TRIANGLE_H
5  #define TRIANGLE_H
6  #include "shape.h"
7
8  // Class definition
9  class Triangle : public Shape
10 {
11     private:
12         double side1;
13         double side2;
14         double side3;
15         bool isValid () const;
16     public:
17         Triangle (double side1, double side2, double side3);
18         ~Triangle ();
19         void print() const;
20         double getArea() const;
```

Interface File for the Triangle Class Part 2

Program 12.15 *File triangle.h*

```
21         double getPerimeter() const;  
22     }  
23 #endif
```

Implementation File the Triangle Class

Part 1

Program 12.16 File triangle.cpp

```
1  /*****
2  * The implementation file the Triangle class *
3  *****/
4  #include "triangle.h"
5
6  // Constructor
7  Triangle :: Triangle (double s1, double s2, double s3)
8  : side1(s1), side2(s2), side3 (s3)
9  {
10     if (!isValid())
11     {
12         cout << "Invalid triangle!";
13         assert (false);
14     }
15 }
16 // Destructor
17 Triangle :: ~Triangle ()
18 {
19 }
20 // Definition of print function
```

Implementation File the Triangle Class

Part 2

Program 12.16 File triangle.cpp

```
21 void Triangle :: print() const
22 {
23     cout << "Triangle of : " << side1 << " X " << side2 << " X ";
24     cout << side3 << endl;
25 }
26 // Finding the area
27 double Triangle :: getArea() const
28 {
29     double s = (side1 + side2 + side3) / 2;
30     return (sqrt (s * (s - side1) * (s - side2) * (s - side3)));
31 }
32 // Finding the perimeter
33 double Triangle :: getPerimeter() const
34 {
35     return (side1 + side2 + side3);
36 }
37 // Private isValid function
38 bool Triangle :: isValid () const
39 {
40     bool fact1 = (side1 + side2) > side3;
```

Implementation File the Triangle Class

Part 3

Program 12.16 *File triangle.cpp*

```
41     bool fact2 = (side1 + side3) > side2;  
42     bool fact3 = (side2 + side3) > side1;  
43     return (fact1 && fact2 && fact3);  
44 }
```

Interface File for the Circle Class Part 1

Program 12.17 File circle.h

```
1  /*****
2  * The interface file for the Circle class          *
3  *****/
4  #ifndef CIRCLE_H
5  #define CIRCLE_H
6  #include "shape.h"
7
8  // Class definition
9  class Circle : public Shape
10 {
11     private:
12         double radius;
13         bool isValid () const;
14     public:
15         Circle (double radius);
16         ~Circle ();
17         void print() const;
18         double getArea() const;
19         double getPerimeter() const;
20 };
```

Interface File for the Circle Class Part 2

Program 12.17 *File circle.h*

```
21 #endif
```


Implementation File the Circle Class Part 1

Program 12.18 File circle.cpp

```
1  /*****
2  * The implementation file the Circle class *
3  *****/
4  #include "circle.h"
5
6  // Constructor
7  Circle :: Circle (double r)
8  : radius (r)
9  {
10     if (!isValid())
11     {
12         cout << "Invalid circle!";
13         assert (false);
14     }
15 }
16 // Destructor
17 Circle :: ~Circle ()
18 {
19 }
20 // Definition of print function
```

Implementation File the Circle Class Part 2

Program 12.18 File circle.cpp

```
21 void Circle :: print() const
22 {
23     cout << "Circle of radius : " << radius << endl;
24 }
25 // Finding the area
26 double Circle :: getArea() const
27 {
28     return (3.14 * radius * radius);
29 }
30 // Finding the perimeter
31 double Circle :: getPerimeter() const
32 {
33     return 2 * 3.14 * radius;
34 }
35 // Private isValid function
36 bool Circle :: isValid () const
37 {
38     return (radius > 0);
39 }
```

Interface File for the Ellipse Class Part 1

Program 12.19 File ellipse.h

```
1  /*****
2  * The interface file for the Ellipse class *
3  *****/
4  #ifndef ELLIPSE_H
5  #define ELLIPSE_H
6  #include "shape.h"
7
8  // Class definition
9  class Ellipse : public Shape
10 {
11     private:
12         double radius1;
13         double radius2;
14         bool isValid () const;
15     public:
16         Ellipse (double radius1, double radius2);
17         ~Ellipse ();
18         void print() const;
19         double getArea () const;
20         double getPerimeter () const;
```

Interface File for the Ellipse Class Part 2

Program 12.19 *File ellipse.h*

```
21 }  
22 #endif
```

Implementation File for the Ellipse Class

Part 1

Program 12.20 File *ellipse.cpp*

```
1  /*****
2  * The interface file for the Ellipse class
3  *****/
4  #include "ellipse.h"
5
6  // Constructor
7  Ellipse :: Ellipse (double r1, double r2)
8  : radius1 (r1), radius2 (r2)
9  {
10     if (!isValid())
11     {
12         cout << "Invalid ellipse!";
13         assert (false);
14     }
15 }
16 // Destructor
17 Ellipse :: ~Ellipse ()
18 {
19 }
20 // Definition of print function
```

Implementation File for the Ellipse Class

Part 2

Program 12.20 File *ellipse.cpp*

```
21 void Ellipse :: print() const
22 {
23     cout << "Ellipse of radii: " << radius1 << " X " <<
24     cout << radius2 << endl;
25 }
26 // Finding the area
27 double Ellipse :: getArea () const
28 {
29     return (3.14 * radius1 * radius2);
30 }
31 // Finding the perimeter
32 double Ellipse ::getPerimeter () const
33 {
34     double temp = (radius1 * radius1 + radius2 * radius2) / 2;
35     return (2 * 3.14 * temp);
36 }
37 // Private isValid function
38 bool Ellipse :: isValid () const
39 {
40     return (radius1 > 0 && radius2 > 0);
```

Implementation File for the Ellipse Class

Part 3

Program 12.20 *File ellipse.cpp*

```
41 }
```

Application File to Test All Classes Part 1

Program 12.21 File *app.cpp*

```
1  /*****
2  * The application file to test all classes *
3  *****/
4  #include "square.h"
5  #include "rectangle.h"
6  #include "triangle.h"
7  #include "circle.h"
8  #include "ellipse.h"
9
10 int main ( )
11 {
12     // Instantiation and testing the Square class
13     cout << "Information about a square" << endl;
14     Square square (5);
15     square.print ();
16     cout << "area: " << square.getArea () << endl;
17     cout << "Perimeter: " << square.getPerimeter () << endl;
18     cout << endl;
19     // Instantiation and testing the Rectangle class
20     cout << "Information about a rectangle" << endl;
```


Application File to Test All Classes Part 2

Program 12.21 File *app.cpp*

```
21     Rectangle rectangle (5, 4);
22     rectangle.print ();
23     cout << "area: " << rectangle.getArea () << endl;
24     cout << "Perimeter: " << rectangle.getPerimeter () << endl;
25     cout << endl;
26     // Instantiation and testing the Triangle class
27     cout << "Information about a triangle" << endl;
28     Triangle triangle (3, 4, 5);
29     triangle.print ();
30     cout << "area: " << triangle.getArea () << endl;
31     cout << "Perimeter: " << triangle.getPerimeter () << endl;
32     cout << endl;
33     // Instantiation and testing the Circle class
34     cout << "Information about a circle" << endl;
35     Circle circle (5);
36     circle.print ();
37     cout << "area: " << circle.getArea () << endl;
38     cout << "Perimeter: " << circle.getPerimeter () << endl;
39     cout << endl;
40     // Instantiation and testing the Ellipse class
```

Application File to Test All Classes Part 3

Program 12.21 File *app.cpp*

```
41     cout << "Information about an ellipse" << endl;  
42     Ellipse ellipse (5, 4);  
43     ellipse.print ();  
44     cout << "area: " << ellipse.getArea () << endl;;  
45     cout << "Perimeter: " << ellipse.getPerimeter ()<< endl;  
46     return 0;  
47 }
```

Run:

```
Information about a square  
Square of size 5  
area: 25  
Perimeter: 20
```

```
Information about a rectangle  
Rectangle of 5 X 4  
area: 20  
Perimeter: 18
```

Application File to Test All Classes Part 4

Program 12.21 *File app.cpp*

Information about a triangle

Triangle of : 3 X 4 X 5

area: 6

Perimeter: 12

Information about a circle

Circle of radius : 5

area: 78.5

Perimeter: 31.4

Information about an ellipse

Ellipse of radii: 5 X 4

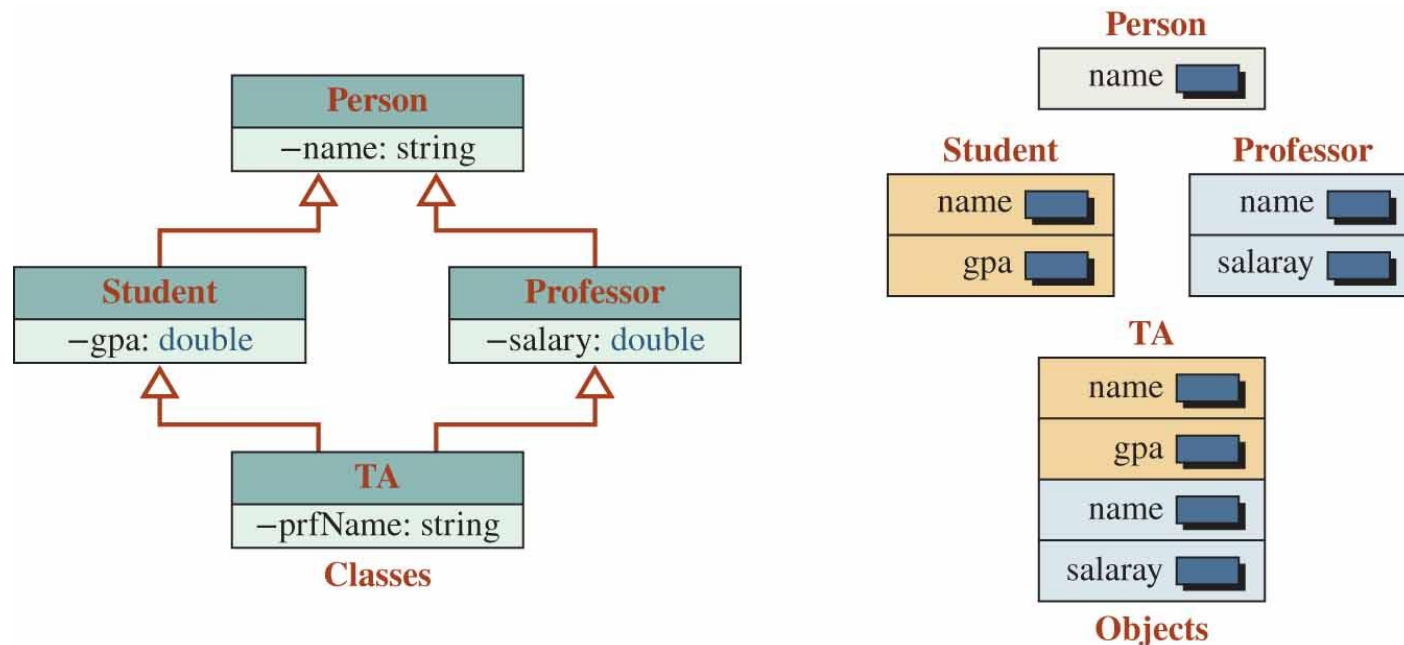
area: 62.8

Perimeter 28.4339

Multiple Inheritance Part 1

C++ allows us to derive a class from more than one class. As a simple example, we can have a class named TA (teaching assistant) that is inherited from two classes: Student and Professor as shown in Figure 12.8.

Figure 12.8 *Classes and objects in multiple inheritance*



Multiple Inheritance Part 2

Unfortunately, the inheritance fails when we code these classes because the TA class inherits the data member name from both Student and Professor class.

Virtual Base in Inheritance₁

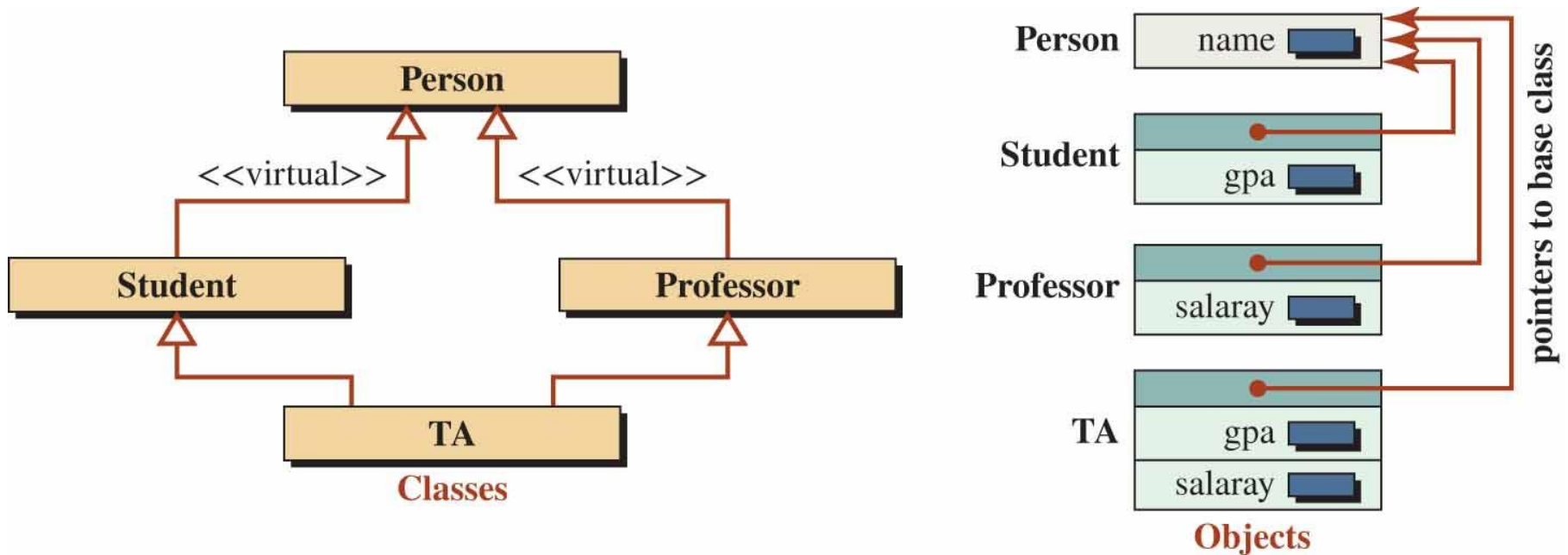
One solution for the problem of duplicated shared data members in multiple inheritance is to use *virtual base inheritance*.

In this type of inheritance, two classes can inherit from a common base using the *virtual* keyword.

Figure 12.9 shows this approach.

Virtual Base in Inheritance₂

Figure 12.9 *Classes and objects in virtual base inheritance*



Virtual Base in Inheritance Part 1

In this case, we have the following four classes.

```
class Person {...};  
class Student: virtual public Person {...};  
class Professor: virtual public Person {...};  
class TA: public Student, public Professor {...};
```

When we use virtual base inheritance, the object of the virtual base class is not stored in each object of the derived class.

The object of the virtual base class is stored separately and each derived class has a pointer to this object.

Virtual Base in Inheritance Part 2

One problem when using the virtual base technique is that we need to avoid *delegation* as discussed in the previous chapter.

In other words, we cannot define a print function in the TA class by calling the corresponding print functions in the Student and Professor class because the data member name would be printed three times.

There is more than a solution to this dilemma; the one that we recommend is to make the common data members protected to be seen in all derived classes and avoid using delegated member functions.

Virtual Base in Inheritance Part 3

When we use virtual base inheritance, the object of the virtual base class is not stored in each object of the derived class.

The object of the virtual base class is stored separately and each derived class has a pointer to this object.

Interface File for the Person Class

Program 12.22 File *person.h*

```
1  /*****
2   * The interface file for the Person class          *
3   *****/
4  #ifndef PERSON_H
5  #define PERSON_H
6  #include <iostream>
7  #include <cassert>
8  using namespace std;
9
10 class Person
11 {
12     protected:
13         string name; // Protected data member
14     public:
15         Person (string name);
16         ~Person ();
17         void print ();
18 };
19 #endif
```

Implementation File for the Person Class₂

Program 12.23 File *person.cpp*

```
1  /*****
2  * The implementation file for the Person class *
3  *****/
4  #include "person.h"
5
6  // Constructor
7  Person :: Person (string nm)
8  : name (nm)
9  {
10 }
11 // Destructor
12 Person :: ~Person ()
13 {
14 }
15 // Print member function
16 void Person :: print ()
17 {
18     cout << "Person" << endl;
19     cout << "Name: " << name << endl << endl;
20 }
```

Interface File for the Student Class

Program 12.24 File student.h

```
1  /*****
2  * The interface file for the Student class          *
3  *****/
4  #ifndef STUDENT_H
5  #define STUDENT_H
6  #include "person.h"
7
8  class Student: virtual public Person // Virtual inheritance
9  {
10     protected:
11         double gpa; // Protected data member
12     public:
13         Student (string name, double gpa);
14         ~Student ();
15         void print ();
16 };
17 #endif
```

Implementation File for the Student Class

Part 1

Program 12.25 File student.cpp

```
1  /*****
2  * The implementation file for the Student class          *
3  *****/
4  #include "Student.h"
5
6  // Constructor
7  Student :: Student (string name, double gp)
8  : Person (name), gpa (gp)
9  {
10     assert (gpa <= 4.0);
11 }
12 // Destructor
13 Student :: ~Student()
14 {
15 }
16 // Print member function uses a protected data member (name)
17 void Student :: print ()
18 {
19     cout << "Student " << endl;
20     cout << "Name: " << name << " ";
```

Implementation File for the Student Class

Part 2

Program 12.25 *File student.cpp*

```
21     cout << "GPA: " << gpa << endl << endl;  
22 }
```

Interface File for the Professor Class

Program 12.26 File professor.h

```
1  /*****
2  * The interface file for the Professor class
3  *****/
4  #ifndef PROFESSOR_H
5  #define PROFESSOR_H
6  #include "person.h"
7
8  class Professor: virtual public Person // Virtual inheritance
9  {
10     protected:
11         double salary; // Protected data member
12     public:
13         Professor (string name, double salary);
14         ~Professor ();
15         void print ();
16 };
17 #endif
```


Implementation File for the Professor Class Part 1

Program 12.27 File professor.cpp

```
1  /*****
2  * The implementation file for the Professor class      *
3  *****/
4  #include "professor.h"
5
6  // Constructor
7  Professor :: Professor (string nm, double sal)
8  : Person (nm), salary (sal)
9  {
10 }
11 // Destructor
12 Professor :: ~Professor ()
13 {
14 }
15 // Print member function
16 void Professor :: print ()
17 {
18     cout << "Professor " << endl;
19     cout << "Name: " << name << " ";
20     cout << "Salary: " << salary << endl << endl;
```

Implementation File for the Professor

Class Part 2

Program 12.27 *File professor.cpp*

```
21 }
```

Interface File for TA Class

Program 12.28 File ta.h

```
1  /*****
2  * The interface file for the TA class
3  *****/
4  #ifndef TA_H
5  #define TA_H
6  #include "student.h"
7  #include "professor.h"
8
9  class TA: public Professor, public Student // Double inheritance
10 {
11     public:
12         TA (string name, double gpa, double sal);
13         ~TA ();
14         void print ();
15 };
16 #endif
```

Implementation File for the TA Class Part 1

Program 12.29 File ta.cpp

```
1  /*****
2   * The implementation file for the TA class *
3   *****/
4  #include "ta.h"
5
6  // Constructor
7  TA :: TA (string nm, double gp, double sal)
8  : Person (nm), Student (nm, gp), Professor (nm, sal)
9  {
10 }
11 // Destructor
12 TA :: ~TA ()
13 {
14 }
15 // Print member function
16 void TA :: print ()
17 {
18     cout << "Teaching Assistance: " << endl;
19     cout << "Name: " << name << " ";
20     cout << "GPA: " << gpa << " ";
```

Implementation File for the TA Class Part 2

Program 12.29 *File ta.cpp*

```
21 cout << "Salary: " << salary << endl << endl;  
22 }
```

Application to Test All Four Classes Part 1

Program 12.30 *File application.cpp*

```
1  /*****
2  * The application to test all four classes (Person, Student, *
3  * Professor, and TA) .                                     *
4  *****/
5  #include "ta.h"
6
7  int main ( )
8  {
9      // Testing Person class
10     Person person ("John");
11     person.print ();
12     // Testing Student class
13     Student student ("Anne", 3.9);
14     student.print ();
15     // Testing Professor class
16     Professor professor ("Lucie", 78000);
17     professor.print ();
18     // Testing TA class
19     TA ta ("George", 3.2, 20000);
20     ta.print ();
```

Application to Test All Four Classes Part 2

Program 12.30 *File application.cpp*

```
21     return 0;  
22 }
```

Run:

Person

Name: John

Student

Name: Anne GPA: 3.9

Professor

Name: Lucie Salary: 78000

Teaching Assistance:

Name: George GPA: 3.2 Salary: 20000

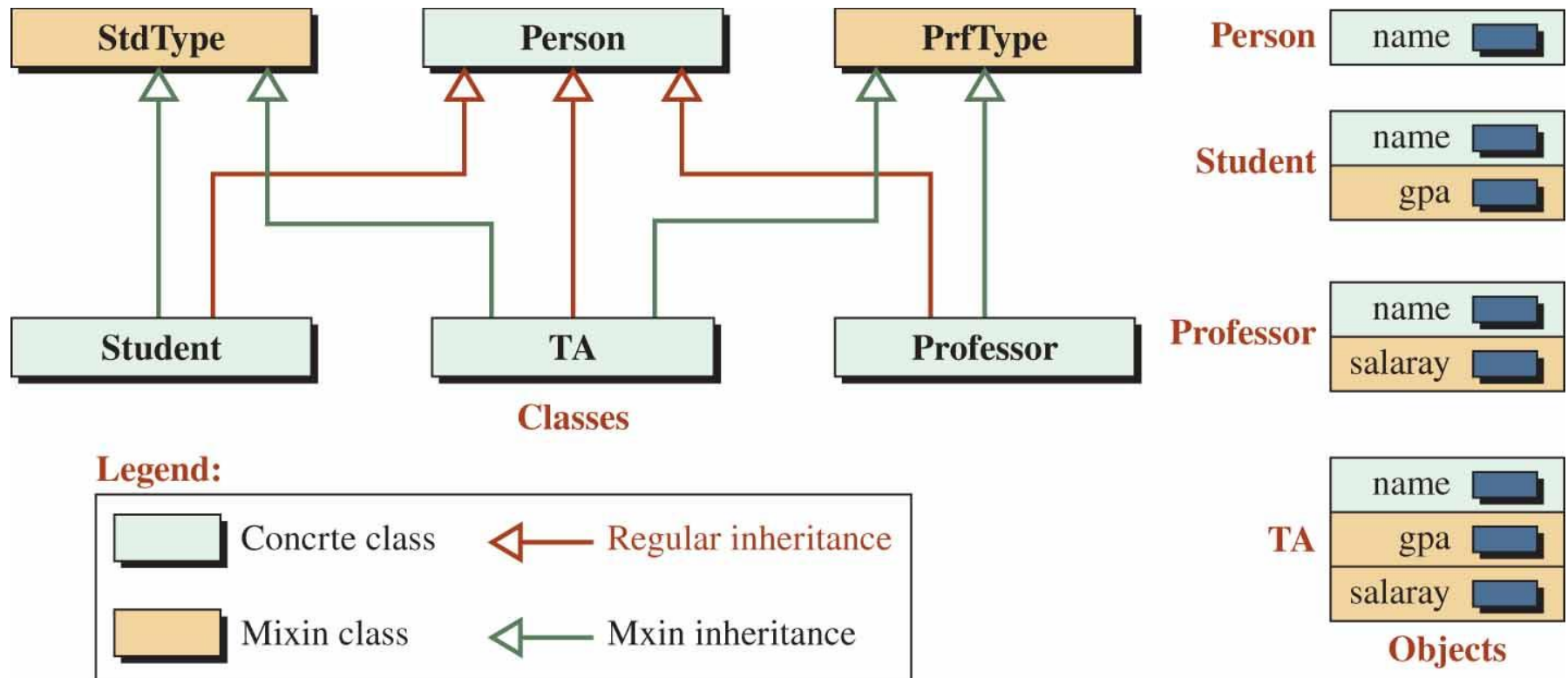
Mixin Classes

Another solution for the problem of common base classes in multiple inheritance is the use of mixin classes.

A mixin class is never instantiated (it has some pure virtual functions), but it can add data members to other classes.

Multiple Inheritance Using mixin Classes

Figure 12.10 *Multiple inheritance using mixin classes*



Interface File for StdType Abstract Class

Program 12.31 File stdtype.h

```
1  /*****
2   * The interface file for StdType abstract class          *
3   *****/
4  #ifndef STDTYPE_H
5  #define STDTYPE_H
6  #include <iostream>
7  using namespace std;
8
9  class StdType
10 {
11     protected:
12         double gpa;
13     public:
14         virtual void printGPA( ) = 0 ;
15 };
16 #endif
```

Interface File for PrfType Abstract Class

Program 12.32 File prftype.h

```
1  /*****
2  * The interface file for PrfType abstract class *
3  *****/
4  #ifndef PRFTYPE_H
5  #define PRFTYPE_H
6  #include <iostream>
7  using namespace std;
8
9  class PrfType
10 {
11     protected:
12         double salary;
13     public:
14         virtual void printSalary () = 0;
15 };
16 #endif
```

Interface File for Person Concrete Class

Program 12.33 File *person.h*

```
1  /*****
2   * The interface file for Person concrete class      *
3   *****/
4  #ifndef PERSON_H
5  #define PERSON_H
6  #include <iostream>
7  #include <string>
8  #include <iomanip>
9  using namespace std;
10
11 class Person
12 {
13     private:
14         string name;
15     public:
16         Person (string name);
17         void print ();
18 };
19 #endif
```

Interface File for Student Concrete Class

Program 12.34 File student.h

```
1  /*****
2  * The interface file for Student concrete class. This class *
3  * inherits from two classes: Person and StdType.          *
4  *****/
5  #ifndef STUDENT_H
6  #define STUDENT_H
7  #include "person.h"
8  #include "stdtype.h"
9
10 class Student: public Person, public StdType
11 {
12     public:
13         Student (string name, double gpa);
14         void printGPA();
15         void print();
16 };
17 #endif
```

Interface File for Professor Concrete Class

Program 12.35 File professor.h

```
1  /*****
2  * The interface file for Professor concrete class. This class *
3  * inherits from two classes: Person and PrfType.           *
4  *****/
5  #ifndef PROFESSOR_H
6  #define PROFESSOR_H
7  #include "person.h"
8  #include "prftype.h"
9
10 class Professor : public Person, public PrfType
11 {
12     public:
13         Professor (string name, double salary);
14         void printSalary();
15         void print ();
16 };
17 #endif
```

Interface File for TA Concrete Class

Program 12.36 File ta.h

```
1  /*****
2  * The interface file for TA concrete class. This class      *
3  * inherits from tree classes: Person and StdType and PrfType. *
4  *****/
5  #ifndef TA_H
6  #define TA_H
7  #include "person.h"
8  #include "stdtype.h"
9  #include "prftype.h"
10
11 class TA: public Person, public StdType, public PrfType
12 {
13     public:
14         TA (string name, double gpa, double salary);
15         void printGPA ();
16         void printSalary();
17         void print ();
18 };
19 #endif
```

Implementation File for Person Concrete Class

Program 12.37 File *person.cpp*

```
1  /*****
2   * The implementation file for Person concrete class          *
3   *****/
4  #include "person.h"
5
6  // Constructor
7  Person :: Person (string nm)
8  : name (nm)
9  {
10 }
11 // Print member function
12 void Person :: print ( )
13 {
14     cout << "Name: " << name << endl;
15 }
```


Implementation file for Student Concrete Class 1

Program 12.38 File student.cpp

```
1  /*****
2   * The implementation file for Student concrete class      *
3   *****/
4  #include "student.h"
5
6  // Constructor
7  Student :: Student (string na, double gp)
8  :Person (na)
9  {
10     gpa = gp; // Assignment, not initialization
11 }
12 // PrintGPA member function
13 void Student :: printGPA ()
14 {
15     cout << "GPA: " << fixed << setprecision (2) << gpa << endl;
16 }
17 // Print member function
18 void Student :: print ()
19 {
20     Person :: print();
```

Implementation file for Student Concrete Class 2

Program 12.38 *File student.cpp*

```
21     printGPA ();  
22 }
```

Implementation File for Professor Concrete Class 1

Program 12.39 File professor.cpp

```
1  /*****
2   * The implementation file for Professor concrete class      *
3   *****/
4  #include "professor.h"
5
6  // Constructor
7  Professor :: Professor (string nm, double sal)
8  : Person (nm)
9  {
10     salary = sal; // Assignment, not initialization
11 }
12 // PrintSalary member function
13 void Professor :: printSalary ()
14 {
15     cout << "Salary: ";
16     cout << fixed << setprecision (2) << salary << endl;
17 }
18 // General print function
19 void Professor :: print ()
20 {
```

Implementation File for Professor Concrete Class 2

Program 12.39 *File professor.cpp*

```
21     Person :: print() ;  
22     printSalary() ;  
23 }
```

Implementation File for TA Concrete Class

Part 1

Program 12.40 File ta.cpp

```
1  /*****
2   * The implementation file for TA concrete class          *
3   *****/
4  #include "ta.h"
5
6  // Constructor
7  TA :: TA (string nm, double gp, double sal)
8  : Person (nm)
9  {
10     gpa = gp; // Assignment, not initialization
11     salary = sal; // Assignment, not initialization
12 }
13 // member function to print GPA
14 void TA :: printGPA ()
15 {
16     cout << "GPA: " << gpa << endl;
17 }
18 // member function to print salary
19 void TA :: printSalary ()
20 {
```

Implementation File for TA Concrete Class

Part 2

Program 12.40 File *ta.cpp*

```
21     cout << "Salary: ";
22     cout << fixed << setprecision (2) << salary << endl;
23 }
24 // General print function
25 void TA :: print ()
26 {
27     Person :: print();
28     printGPA ();
29     printSalary();
30 }
```

Application File to Test the Three Classes

Part 1

Program 12.41 *File application.cpp*

```
1  /*****
2  * The application file to test the three classes          *
3  *****/
4  #include "student.h"
5  #include "professor.h"
6  #include "ta.h"
7
8  int main ( )
9  {
10     // Instantiation of four objects
11     Person per ("John");
12     Student std ("Linda", 3.9);
13     Professor prf("George", 89000);
14     TA ta ("Lucien", 3.8, 23000);
15     // Printing information about a person
16     cout << "Information about person" << endl;
17     per.print();
18     cout << endl << endl;
19     // Printing information about a student
20     cout << "Information about student" << endl;
```

Application File to Test the Three Classes

Part 2

Program 12.41 *File application.cpp*

```
21     std::print ();  
22     cout << endl << endl;  
23     // Printing information about a professor  
24     cout << "Information about professor" << endl;  
25     prf.print();  
26     cout << endl << endl;  
27     // Printing information about a teaching assistant  
28     cout << "Information about teaching assistance " << endl;  
29     ta.print();  
30     cout << endl << endl;  
31     return 0;  
32 }
```

Run:

Information about person

Name: John

Information about student

Name: Linda

GPA: 3.90

Application File to Test the Three Classes

Part 3

Program 12.41 *File application.cpp*

Run:

Information about professor

Name: George

Salary: 89000.00

Information about teaching assistance

Name: Lucien

GPA: 3.80

Salary: 23000.00